Testing as Requirements Definition

In Chapter 3 the tests were ours. We wrote them to find out what a converter should do, to catch our own mistakes, and to keep early cases working as the code grew. A teammate could read them, because a teammate reads code.

A customer usually cannot. Sometimes the customer is exactly the person who has to say what the software should do. This chapter asks what a test looks like when it is also the written requirement: something the customer can read, correct, and agree to, and something the computer can still run.

A requirement as a test

Start with a requirement small enough to say in one sentence: *the Wikipedia page on light bulbs mentions Edison.* Chapter 4 gave us the tools to check it. The simplest check is to look at the text of the page. Here it is as a pytest test that drives Chrome the way a person would, by searching on Wikipedia's home page (1):

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC


def test_light_bulb_mentions_edison():
    browser = webdriver.Chrome()
    try:
        browser.get("https://en.wikipedia.org/wiki/Main_Page")
        wait = WebDriverWait(browser, 15)
        search = wait.until(EC.element_to_be_clickable((By.NAME, "search")))
        start = browser.current_url
        search.send_keys("light bulb", Keys.ENTER)
        wait.until(EC.url_changes(start))
        wait.until(EC.visibility_of_element_located((By.ID, "firstHeading")))
        text = browser.find_element(By.TAG_NAME, "body").text
        assert "Edison" in text
    finally:
        browser.quit()


def main():
    test_light_bulb_mentions_edison()


if __name__ == "__main__":
    main()
```

The test does its job. Now read it as the customer would. The requirement was one sentence. The test is nearly thirty lines of Python, Selenium, and Chrome details: a search box found by name, a key press, an explicit wait, and a body element. A customer cannot tell from it whether we understood the requirement. Only a programmer can look at it and say that it matches the sentence.

That is fine for tests we write for ourselves. It is a poor way to communicate with someone who is not a programmer. The next step keeps the programmer's code and puts the sentence back in front.

Sentences backed by code

Here is the same requirement written as a scenario:

```
Feature: Inventors on Wikipedia

  Scenario: The light bulb page mentions Edison
    Given I have opened the Wikipedia home page
    When I search for Light Bulb
    Then the page text contains Edison
```

Each line is a **clause**. The **Given** clause states a precondition: where we start. The **When** clause is an action: what someone does. The **Then** clause is a result: what we expect to see. This is almost English. It follows the rules of a small language called **Gherkin** (3), which makes it possible for a program to read it.

A customer can read these three lines and say whether they describe what he wants. If the customer can explain each clause in plain words, a developer can implement each clause as faithfully as possible. The tool we use, **behave** (2), reads the scenario, finds the Python function that matches each clause, and runs them in order. The pieces live in two places:

```
features/
  light_bulb.feature
  steps/
    wikipedia_steps.py
```

The `.feature` file holds the sentences. The files in `steps/` hold the code that carries them out. Each function there is called a **step**.

Implementing the clauses

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC


@given("I have opened the Wikipedia home page")
def open_home_page(context):
    # Set up the precondition...
    context.browser = webdriver.Chrome()
    context.add_cleanup(context.browser.quit)
    context.wait = WebDriverWait(context.browser, 15)
    context.browser.get("https://en.wikipedia.org/wiki/Main_Page")
    # ...and verify it: the home page is up and its search box is usable.
    context.wait.until(EC.element_to_be_clickable((By.NAME, "search")))


@when("I search for Light Bulb")
def search_for_light_bulb(context):
    start = context.browser.current_url
    box = context.browser.find_element(By.NAME, "search")
    box.send_keys("Light Bulb", Keys.ENTER)
    # Verify the action was taken: we left the home page for an article.
    context.wait.until(EC.url_changes(start))
    heading = context.browser.find_element(By.ID, "firstHeading").text
    assert not heading.startswith("Search results"), heading


@then("the page text contains Edison")
def page_text_contains_edison(context):
    text = context.browser.find_element(By.TAG_NAME, "body").text
    assert "Edison" in text
```

The decorator on each function holds the text of the clause it implements. When behave reads a line from the feature file, it finds the step whose text matches and calls it.

The steps need to share things. The browser started in the Given must still be there in the When and the Then. behave passes every step the same `context` object, and anything we store on it stays available for the rest of the scenario. The Given stores `context.browser` and `context.wait`. The later steps use them. The line with `add_cleanup` closes the browser when the scenario ends. We will come back to it.

Each clause has its own job, and the job has two halves:

- **Given** sets up the precondition and verifies it. We open the home page, then confirm the search box is ready to use.
- **When** takes the action and verifies that the action happened. We search, then confirm that the browser left the home page and landed on an article, not a list of search results. Waiting for the address to change matters. The page we start on may already contain the element we would wait for, and a check for it could pass before the search finished.
- **Then** verifies the result and does nothing else. The page text contains Edison.

Keeping those jobs apart makes a failure easier to read. If the Given fails, the starting point was wrong. If the When fails, the action did not work. If only the Then fails, the requirement itself is not met.

The step functions are code, and code can be wrong. We test them too, but those tests sit at a lower level than the requirement, the way the converter tests in Chapter 3 sat beneath any business need for numbers. The scenario is the statement the customer agrees to. The step implementations are ours.

Reusing clauses

Suppose the customer adds a second requirement: the page for helicopters should mention Sikorsky. We add a second scenario to the feature file:

```
Feature: Inventors on Wikipedia

  Scenario: The light bulb page mentions Edison
    Given I have opened the Wikipedia home page
    When I search for Light Bulb
    Then the page text contains Edison

  Scenario: The helicopter page mentions Sikorsky
    Given I have opened the Wikipedia home page
    When I search for Helicopter
    Then the page text contains Sikorsky
```

The Given clause is identical, so its step is reused. The other two clauses differ only in a word. Writing a new step for each would give us two functions that are almost the same. We can instead let the clause carry a parameter:

```
@when("I search for {invention}")
def search_for_invention(context, invention):
    ...

@then("the page text contains {inventor}")
def page_text_contains_inventor(context, inventor):
    ...
```

The text in braces matches whatever words appear there in the feature file, and the step receives it as an argument. One step now handles any invention and any inventor.

Arguments arrive as text, but they can be converted. A type after the name asks for it. A step written as `{count:d}` receives an `int`, and `{price:f}` receives a `float` (4):

```
@then("the page text mentions {name} at least {count:d} times")
def mentions_at_least(context, name, count):
    text = context.browser.find_element(By.TAG_NAME, "body").text
    assert text.count(name) >= count
```

If the text does not fit the type, the clause does not match, and behave reports that no step is defined. That is easier to understand than an error buried in our own code.

The Given can be made general in the same way. `Given I have opened {location}` looks up a readable name such as “the Wikipedia home page” in a small dictionary and falls back to treating the text as an address. The same clause then works for any location.

Many examples, one outline

What if there are ten inventions to check? We do not need ten scenarios. A **scenario outline** writes the clauses once, with placeholders in angle brackets, and an **examples** table supplies the values. behave runs the whole scenario once for each row (5):

```
Feature: Inventors on Wikipedia

  Scenario Outline: An invention's page names its inventor
    Given I have opened the Wikipedia home page
    When I search for <invention>
    Then the page text contains <inventor>

    Examples: Communication
      | invention  | inventor |
      | Telephone  | Bell     |
      | Radio      | Marconi  |
      | Telegraphy | Morse    |

    Examples: Recording and light
      | invention  | inventor |
      | Phonograph | Edison   |
      | Light Bulb | Edison   |

    Examples: Flight
      | invention  | inventor |
      | Helicopter | Sikorsky |
      | Airplane   | Wright   |

    Examples: Industry
      | invention      | inventor  |
      | Printing Press | Gutenberg |
      | Steam Engine   | Watt      |
      | Dynamite       | Nobel     |
```

Adding a case means adding a row. The code does not change. The table is also something a customer can review quickly, and he can propose a new row without knowing anything about Python.

An outline may have several examples tables, each with its own name (5). Here they group inventions by kind. The grouping changes nothing about how the tests run. It helps a person scanning the table find the row he cares about.

The steps now look like this:

```
LOCATIONS = {
    "the Wikipedia home page": "https://en.wikipedia.org/wiki/Main_Page",
}


@given("I have opened {location}")
def open_location(context, location):
    # Set up the precondition: a named place, or any URL, opened in Chrome.
    url = LOCATIONS.get(location, location)
    context.browser = webdriver.Chrome()
    context.add_cleanup(context.browser.quit)
    context.wait = WebDriverWait(context.browser, 15)
    context.browser.get(url)
    # Verify it: the browser loaded a real page.
    assert context.browser.title, "the page has no title"


@when("I search for {invention}")
def search_for_invention(context, invention):
    start = context.browser.current_url
    box = context.browser.find_element(By.NAME, "search")
    box.send_keys(invention, Keys.ENTER)
    # Verify the action was taken: we left the home page for an article.
    context.wait.until(EC.url_changes(start))
    heading = context.browser.find_element(By.ID, "firstHeading").text
    assert not heading.startswith("Search results"), heading


@then("the page text contains {inventor}")
def page_text_contains_inventor(context, inventor):
    text = context.browser.find_element(By.TAG_NAME, "body").text
    assert inventor in text
```

Setting up and tearing down

Every scenario here needs a browser, so something has to start it and something has to close it. There are several places to do that, and they differ in what they cost and who can see them.

In the steps. We have been doing this already. The Given starts the browser, and `add_cleanup` registers a function to close it when the scenario's part of the context is removed (6). Everything a scenario stores on the context belongs to that scenario, so each row of an outline gets its own fresh browser. That is slow but safe, since nothing carries over from one row to the next.

In a Background. When every scenario in a feature starts the same way, the Given can move out of the scenarios and into a **Background**, one step above a set of outlines (5). Here is a feature with two outlines that share one:

```
Feature: Inventors on Wikipedia

  Background:
    Given I have opened the Wikipedia home page

  Scenario Outline: An invention's page names its inventor
    When I search for <invention>
    Then the page text contains <inventor>

    # ten rows, as in the previous example

  Scenario Outline: An inventor's page names their invention
    When I search for <inventor>
    Then the page text contains <invention>

    Examples:
      | inventor              | invention      |
      | Thomas Edison         | phonograph     |
      | Igor Sikorsky         | helicopter     |
      | Alexander Graham Bell | telephone      |
      | Johannes Gutenberg    | printing press |
      | Alfred Nobel          | dynamite       |
```

The second outline runs the other way. It searches for a person and looks for his invention. The two outlines use the same two clauses, which is why the step parameters have neutral names like `subject` and `text` instead of `invention` and `inventor`. A background runs before each scenario in the feature, not once for the whole feature. Every row still gets a fresh start.

In hooks. A file named `features/environment.py` can define functions that behave calls automatically. The hooks run around the whole test run, around each feature, around each scenario, and around each step. There are also hooks for tags (6). Nothing in the feature file mentions them, so the customer never sees them:

```python
import os
import re

from selenium import webdriver
from selenium.webdriver.support.ui import WebDriverWait


def before_all(context):
    # Once per run: start one browser for every scenario to share.
    context.browser = webdriver.Chrome()
    context.wait = WebDriverWait(context.browser, 15)


def before_scenario(context, scenario):
    # Once per scenario: reset what a scenario could leave behind.
    context.browser.delete_all_cookies()


def after_scenario(context, scenario):
    # Teardown can look at the outcome. Keep a picture of failures.
    if scenario.status == "failed":
        name = re.sub(r"\W+", "-", scenario.name).strip("-").lower()
        os.makedirs("screenshots", exist_ok=True)
        context.browser.save_screenshot(f"screenshots/{name}.png")


def after_all(context):
    context.browser.quit()
```

Here one browser starts for the whole run and quits at the end. `before_scenario` clears cookies so one scenario cannot leave something behind for the next. `after_scenario` looks at the outcome and saves a screenshot only when the scenario failed. A hook can see whether a scenario passed, and that is useful evidence for tear down.

Where should a given piece of setup go? A few questions decide it:

Place	Runs	Customer sees it?	Good for
Given in a scenario or outline	Each scenario	Yes	A starting point that belongs to these examples
Background	Before each scenario	Yes	A starting point shared by every scenario in a feature
`before_scenario` and `after_scenario`	Each scenario	No	Resetting state; keeping evidence of a failure
`before_feature` and `after_feature`	Once per feature	No	Costly setup that a feature's scenarios can share
`before_all` and `after_all`	Once per run	No	Starting a browser; reading configuration
`add_cleanup` in a step	When the scenario ends	No	Cleaning up right beside the setup that needs it

The customer should see what is true at the start of the examples. Plumbing that has nothing to do with the requirement belongs where he does not have to read it.

There is a tradeoff behind the choice of browser. One browser for the whole run is fast, but the current page, cookies, and anything else the browser remembers can leak from one scenario into the next. A fresh browser for every scenario is slow, but each scenario is independent.

When parsing is the product

In Chapter 3 the converter was a detail. A customer who wants numbers read from text did not care how we handled a leading sign. Now suppose the customer is designing a spreadsheet of his own, with his own ideas about what a cell is. For him, what happens when someone types `5.` or `1e3` into a cell is not a detail. It is the product.

We can reuse the converter from Chapter 3 and put its behavior in the customer's words:

```
Feature: Numbers typed into spreadsheet cells
  My spreadsheet lives or dies on reading what I type the way I mean it.
  A cell that holds a number must hold exactly that number, and a cell
  that does not hold a number must say so instead of guessing.

  Scenario Outline: A cell that holds a number
    When I type "<entry>" into a cell
    Then the cell holds the number <number>

    Examples: Whole numbers
      | entry | number |
      | 7     | 7      |
      | 0     | 0      |
      | 42    | 42     |
      | 105   | 105    |
      | 007   | 7      |

    # Decimals and Signs tables follow

  Scenario Outline: A cell that does not hold a number
    When I type "<entry>" into a cell
    Then the cell is rejected as not a number

    Examples: Not numbers
      | entry |
      | abc   |
      | 12x   |
      | 1.2.3 |
      | +     |
      | -     |
      | .     |
      | --7   |
      | 1e3   |
```

The feature file begins with a description in the customer's own voice. The accepted entries are grouped into named tables of whole numbers, decimals, and signs. The entries that are not numbers get their own outline. The steps call the converter directly, so there is no browser at all:

```
import math

from behave import when, then

from cells import parse


@when('I type "{entry}" into a cell')
def type_into_cell(context, entry):
    # Do the action and keep what happened: a value or an error.
    context.entry = entry
    context.value = None
    context.error = None
    try:
        context.value = parse(entry)
    except ValueError as error:
        context.error = error


@then("the cell holds the number {number}")
def cell_holds_number(context, number):
    assert context.error is None, f"{context.entry!r} was rejected"
    expected = float(number)
    assert math.isclose(context.value, expected, rel_tol=1e-12, abs_tol=1e-12), \
        f"{context.entry!r} gave {context.value!r}, expected {expected!r}"


@then("the cell is rejected as not a number")
def cell_is_rejected(context):
    assert context.error is not None, \
        f"{context.entry!r} was accepted as {context.value!r}"
```

The When keeps whatever happened, a value or an error, on the context. Each Then checks one of them.

Putting the examples in front of a customer brings up decisions that the Chapter 3 tests quietly made for us. The converter rejects `" 7"`, a number with a space in front. Would the customer expect it to work? An empty cell is rejected by the converter, but in a spreadsheet an empty cell is not an error. And does `1e3` mean one thousand? Those are not our decisions. A scenario can hold the answers once the customer gives them, and a new row records each decision. Until he answers, it is better to leave the question open than to guess.

Working through the example

The files in `code/` are checkpoints. The first two are the pytest tests from the start of the chapter. The numbered folders are small behave projects. Install the requirements in `requirements.txt` and set up Chrome as in Chapter 4. Run `pytest` on a file, or run `behave` from inside a numbered folder.

File or folder	Idea introduced
`test_light_bulb_search.py`	The requirement as a browser test written for programmers
`01-direct`	One scenario, with clauses implemented one for one
`02-helicopter`	A second scenario that reuses the Given and repeats the other clauses
`03-reusable`	Parameters, and a scenario outline with a table
`04-examples-table`	A general Given, and ten rows in four named tables
`05-background`	One Background above two outlines
`06-hooks`	Setup and tear down in `environment.py`
`07-spreadsheet-parser`	The Chapter 3 converter, described in the customer's words

Read each checkpoint after trying the change yourself. For an extension, add a row that you suspect will fail. Check what the failure message tells you before you change anything.

A little history

The approach in this chapter has a name: **behavior-driven development**, or **BDD**. It grew out of test-driven development and keeps the same rhythm of examples first, code second. What it changes is who can read the examples (7).

The **Given**, **When**, **Then** layout was borrowed from the way user stories were written for object-oriented analysis and design. Given a starting situation, when something happens, then we expect a result (7).

Cucumber is the best-known tool built on that layout. It was written first in Ruby, and Aslak Hellesøy is among its developers. It reads scenarios written in a plain-text language called **Gherkin**, designed so that customers and business staff can read, and help write, a description of what the software should do (8, 3). A gherkin is a small pickled cucumber, so the vegetable theme carries through.

Cucumber has since been implemented for other programming languages, including Java and JavaScript (8). **SpecFlow** brought the same idea to C# and the .NET platform as a port of Cucumber (9). It has since been superseded by a successor project named Reqnroll (8, 9), so check which tool is current before choosing one for new work. The tool we used, **behave**, does the job for Python: plain-language scenarios, with the implementation written in Python (2). The pattern is the same in all of them. The plain text stays on one side, and the code that carries it out stays on the other.

JavaScript took a different and related path. Jasmine and Mocha both offer a style built from `describe` and `it` blocks. Jasmine's syntax is similar to that of RSpec, the Ruby testing library, and its `it` is meant to start a sentence (10, 11):

```
describe("Wikipedia's light bulb page", () => {
  it("mentions Edison", () => {
    expect(pageText).toContain("Edison");
  });
});
```

This sketch is only an illustration, and the name of the check function varies from library to library. A test report built from names like these reads almost like a specification. It is almost behavioral, though, not fully. The sentences live inside the code. A programmer writes them, and a programmer is usually the only one who reads them. There is no separate document for the customer to approve. When JavaScript teams want that separation, a Cucumber implementation for JavaScript is available (8).

Choosing the right level

Most of the tests we write are for ourselves. A customer who asks for a spreadsheet that reads numbers has no stake in how the converter handles a leading sign, and `describe` or `it` blocks, or plain unit tests, are a fine home for those checks.

Scenarios in Gherkin earn their cost when the customer needs to say, "Yes, that is what I meant." In the spreadsheet example, parsing is the product. What the cell does with `5.` or `1e3` is a decision the customer owns, and putting it in sentences the customer can read gives that conversation a place to happen.

This extra layer has costs. A sentence can read well and still not express what the customer wanted. `Then the page text contains Edison` passes when he appears once in a footnote. The sentence and the code that carries it out can also drift apart, and only a programmer can notice. Scenarios do not replace talking with the customer. They give the conversation something to point at.

References

1. Selenium project. "WebDriver." https://www.selenium.dev/documentation/webdriver/
2. behave project. "behave documentation." https://behave.readthedocs.io/en/latest/
3. Cucumber project. "Gherkin Reference." https://cucumber.io/docs/gherkin/reference/
4. Python package index. "parse," format specification. https://pypi.org/project/parse/
5. behave project. Gherkin documentation: Background, scenario outlines, and examples. https://behave.readthedocs.io/en/latest/gherkin/
6. behave project. "API Reference": environmental controls and the context object. https://behave.readthedocs.io/en/latest/api/
7. Wikipedia contributors. "Behavior-driven development." https://en.wikipedia.org/wiki/Behavior-driven_development
8. Wikipedia contributors. "Cucumber (software)." https://en.wikipedia.org/wiki/Cucumber_(software)
9. Wikipedia contributors. "SpecFlow." https://en.wikipedia.org/wiki/SpecFlow

10. Wikipedia contributors. “Jasmine (JavaScript testing framework).” https://en.wikipedia.org/wiki/Jasmine_(JavaScript_testing_framework)
11. Wikipedia contributors. “Mocha (JavaScript framework).” https://en.wikipedia.org/wiki/Mocha_(JavaScript_framework)

Questions for Thought

1. Which of your own tests would a customer be unable to read? Which of them would the customer need to read before you could say a requirement was understood?
2. A scenario says `Then the page text contains Edison`. Write two different customer intentions that this sentence would satisfy, and decide how you would tell them apart.
3. Why does Gherkin keep the plain-text scenarios in one file and the code that carries them out in another? What would you lose if they were mixed together?
4. A `describe` and `it` test reads like a sentence. Who writes it, and who reads it? What would the customer need in order to approve it?
5. In the spreadsheet example, which of the rejected entries would you ask the customer about first, and why? Which would you decide yourself?
6. A scenario outline has ten rows in four named tables. When might that grouping help the customer, and when would it only help the programmer?
7. The sentences in a scenario and the code behind them can drift apart. What practice would help you notice, given that the customer cannot read the code?
8. The When clause in the Edison example waits for the address to change. What could go wrong if it waited only for a heading on the page?
9. Which of the three clauses should fail if the search box is missing? Why is it useful for that failure to point at one clause?
10. You have ten inventions to check and a table of ten rows. A customer adds an eleventh. What changes in the code, and what changes in the feature file?
11. A scenario in a Background and a scenario in a `before_scenario` hook both start a browser. When would you put the starting point where the customer can see it, and when would you hide it?
12. In the spreadsheet example, the converter rejects `" 7"`. List two questions you would ask the customer before deciding whether that row should say “accepted” or “rejected”.